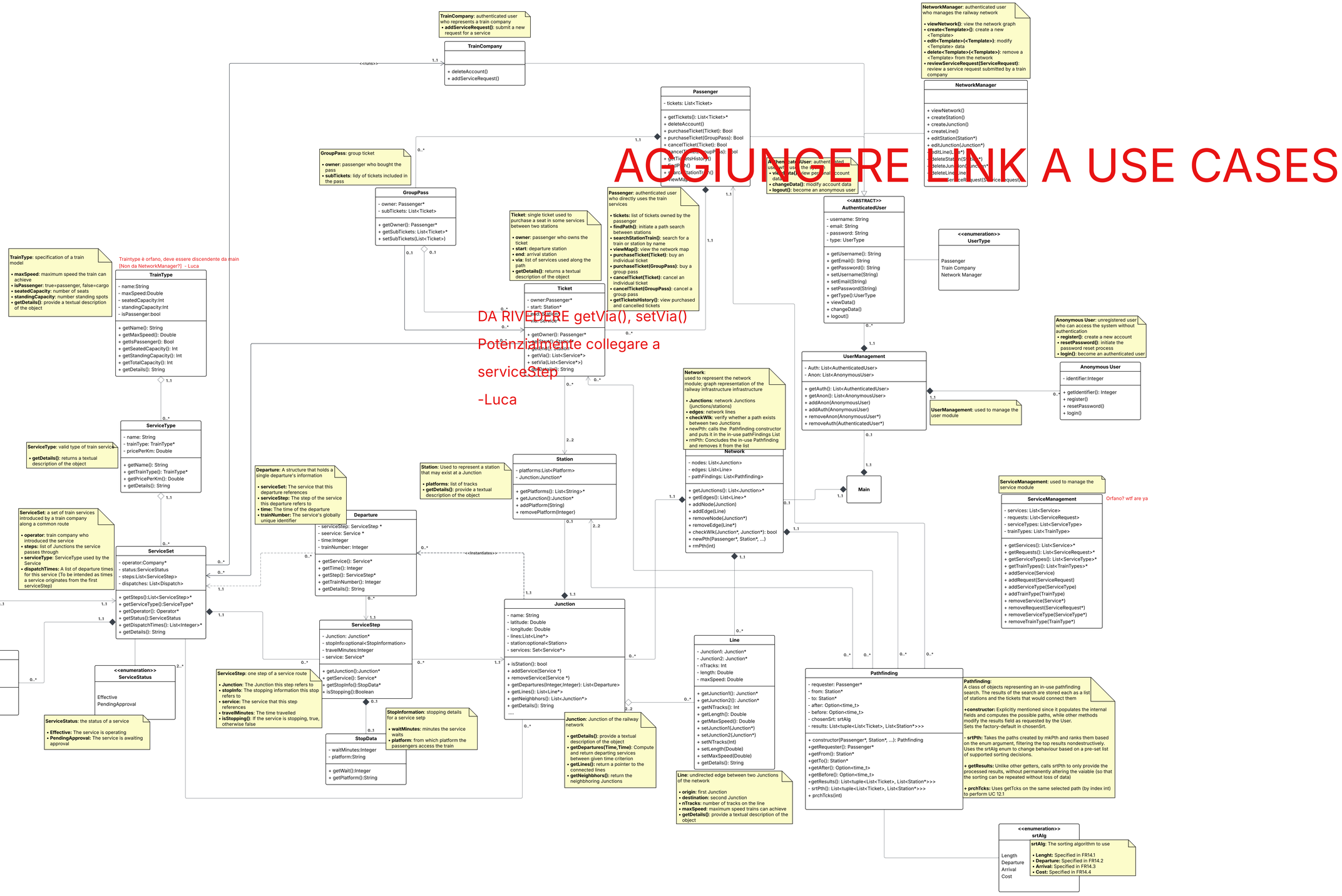
AGGIUNGERE LNK A USE CASES
DA RIVEDERE getVia(), setVia()
Potenzialmente collegare a serviceStep
-Luca
TrainCompany: authenticated user who represents a train company
• addServiceRequest(): submit a new request for a service
TrainCompany
+ deleteAccount()
+ addServiceRequest()
<<runs>>
1..1
NetworkManager: authenticated user who manages the railway network
• viewNetwork(): view the network graph
• create<Template>(): create a new <Template>
• edit<Template>(<Template>): modify <Template> data
• delete<Template>(<Template>): remove a <Template> from the network
• reviewServiceRequest(ServiceRequest): review a service request submitted by a train company
NetworkManager
+ viewNetwork()
+ createStation()
+ createJunction()
+ createLine()
+ editStation(Station*)
+ editJunction(Junction*)
+ editLine(Line*)
+ deleteStation(Station*)
+ deleteJunction(Junction*)
+ deleteLine(Line*)
+ reviewServiceRequest(ServiceRequest*)
Passenger
- tickets: List<Ticket>
+ getTickets(): List<Ticket>*
+ deleteAccount()
+ purchaseTicket(Ticket): Bool
+ purchaseTicket(GroupPass): Bool
+ cancelTicket(Ticket): Bool
+ cancelTicket(GroupPass): Bool
+ getTicketsHistory()
+ findPath()
+ searchStationTrain()
+ viewMap()
GroupPass: group ticket
• owner: passenger who bought the pass
• subTickets: lidy of tickets included in the pass
GroupPass
- owner: Passenger*
- subTickets: List<Ticket>
+ getOwner(): Passenger*
+ getSubTickets: List<Ticket>*
+ setSubTickets(List<Ticket>)
AuthenticatedUser: authenticated user who uses the system
• viewPersonalData(): view personal account data
• changeData(): modify account data
• logout(): become an anonymous user
Passenger: authenticated user who directly uses the train services
• tickets: list of tickets owned by the passenger
• findPath(): initiate a path search between stations
• searchStationTrain(): search for a train or station by name
• viewMap(): view the network map
• purchaseTicket(Ticket): buy an individual ticket
• purchaseTicket(GroupPass): buy a group pass
• cancelTicket(Ticket): cancel an individual ticket
• cancelTicket(GroupPass): cancel a group pass
• getTicketsHistory(): view purchased and cancelled tickets
<<ABSTRACT>>
AuthenticatedUser
- username: String
- email: String
- password: String
- type: UserType
+ getUsername(): String
+ getEmail(): String
+ getPassword(): String
+ setUsername(String)
+ setEmail(String)
+ setPassword(String)
+ getType():UserType
+ viewData()
+ changeData()
+ logout()
<<enumeration>>
UserType
Passenger
Train Company
Network Manager
Ticket: single ticket used to purchase a seat in some services between two stations
• owner: passenger who owns the ticket
• start: departure station
• end: arrival station
• via: list of services used along the path
• getDetails(): returns a textual description of the object
Ticket
- owner:Passenger*
- start: Station*
- end: Station*
- via: Service*
+ getOwner(): Passenger*
+ getStart(): Station*
+ getEnd(): Station*
+ getVia(): List<Service*>
+ setVia(List<Service*>)
+ getDetails(): String
TrainType: specification of a train model
• maxSpeed: maximum speed the train can achieve
• isPassenger: true=passenger, false=cargo
• seatedCapacity: number of seats
• standingCapacity: number standing spots
• getDetails(): provide a textual description of the object
Traintype è orfano, deve essere discendente da main [Non da NetworkManager?] - Luca
TrainType
- name:String
- maxSpeed:Double
- seatedCapacity:Int
- standingCapacity:Int
- isPassenger:bool
+ getName(): String
+ getMaxSpeed(): Double
+ getIsPassenger(): Bool
+ getSeatedCapacity(): Int
+ getStandingCapacity(): Int
+ getTotalCapacity(): Int
+ getDetails(): String
Anonymous User: unregistered user who can access the system without authentication
• register(): create a new account
• resetPassword(): initiate the password reset process
• login(): become an authenticated user
Anonymous User
- identifier:Integer
+ getIdentifier(): Integer
+ register()
+ resetPassword()
+ login()
UserManagement
- Auth: List<AuthenticatedUser>
- Anon: List<AnonymousUser>
+ getAuth(): List<AuthenticatedUser>
+ getAnon(): List<AnonymousUser>
+ addAnon(AnonymousUser)
+ addAuth(AnonymousUser)
+ removeAnon(AnonymousUser*)
+ removeAuth(AuthenticatedUser*)
UserManagement: used to manage the user module
Network: used to represent the network module; graph representation of the railway infrastructure infrastructure
• Junctions: network Junctions (junctions/stations)
• edges: network lines
• checkWlk: verify whether a path exists between two Junctions
• newPth: calls the Pathfinding constructor and puts it in the in-use pathFindings List
• rmPth: Concludes the in-use Pathfinding and removes it from the list
Network
- nodes: List<Junction>
- edges: List<Line>
- pathFindings: List<Pathfinding>
+ getJunctions(): List<Junction>*
+ getEdges(): List<Line>*
+ addNode(Junction)
+ addEdge(Line)
+ removeNode(Junction*)
+ removeEdge(Line*)
+ checkWlk(Junction*, Junction*): bool
+ newPth(Passenger*, Station*, ...)
+ rmPth(int)
Main
ServiceType
- name: String
- trainType: TrainType*
- pricePerKm: Double
+ getName(): String
+ getTrainType(): TrainType*
+ getPricePerKm(): Double
+ getDetails(): String
ServiceType: valid type of train service
• getDetails(): returns a textual description of the object
Departure: A structure that holds a single departure's information
• serviceSet: The service that this departure references
• serviceStep: The step of the service this departure refers to
• time: The time of the departure
• trainNumber: The service's globally unique identifier
Station: Used to represent a station that may exist at a Junction
• platforms: list of tracks
• getDetails(): provide a textual description of the object
Station
- platforms:List<Platform>
- Junction:Junction*
+ getPlatforms(): List<String>*
+ getJunction():Junction*
+ addPlatform(String)
+ removePlatform(Integer)
ServiceManagement: used to manage the service module
Orfano? wtf are ya
ServiceManagement
- services: List<Service>
- requests: List<ServiceRequest>
- serviceTypes: List<ServiceType>
- trainTypes: List<TrainType>
+ getServices(): List<Service>*
+ getRequests(): List<ServiceRequest>*
+ getServiceTypes(): List<ServiceType>*
+ getTrainTypes(): List<TrainTypes>*
+ addService(Service)
+ addRequest(ServiceRequest)
+ addServiceType(ServiceType)
+ addTrainType(TrainType)
+ removeService(Service*)
+ removeRequest(ServiceRequest*)
+ removeServiceType(ServiceType*)
+ removeTrainType(TrainType*)
ServiceSet: a set of train services introduced by a train company along a common route
• operator: train company who introduced the service
• steps: list of Junctions the service passes through
• serviceType: ServiceType used by the Service
• dispatchTimes: A list of departure times for this service (To be intended as times a service originates from the first serviceStep)
ServiceSet
- operator:Company*
- status:ServiceStatus
- steps:List<ServiceStep>
- dispatches: List<Dispatch>
+ getSteps():List<ServiceStep>*
+ getServiceType():ServiceType*
+ getOperator(): Operator*
+ getStatus():ServiceStatus
+ getDispatchTimes(): List<Integer>*
+ getDetails(): String
ServiceRequest: request to introduce a service
• service: service to be requested
• approve(): approve the requested service
• reject(): reject the requested service
ServiceRequest
- service:Service*
+ getId(): Int
+ approve()
+ reject()
Departure
- serviceStep: ServiceStep *
- seervice: Service *
- time:Integer
- trainNumber: Integer
+ getService(): Service*
+ getTime(): Integer
+ getStep(): ServiceStep*
+ getTrainNumber(): Integer
+ getDetails(): String
<<Instantiates>>
ServiceStep
- Junction: Junction*
- stopInfo:optional<StopInformation>
- travelMinutes:Integer
- service: Service*
+ getJunction():Junction*
+ getService(): Service*
+ getStopInfo():StopData*
+ isStopping():Boolean
Junction
- name: String
- latitude: Double
- longitude: Double
- lines:List<Line*>
- station:optional<Station>
- services: Set<Service*>
+ isStation(): bool
+ addService(Service *)
+ removeService(Service *)
+ getDepartures(Integer,Integer): List<Departure>
+ getLines(): List<Line*>
+ getNeighbors(): List<Junction*>
+ getDetails(): String
....
Line
- Junction1: Junction*
- Junction2: Junction*
- nTracks: Int
- length: Double
- maxSpeed: Double
+ getJunction1(): Junction*
+ getJunction2(): Junction*
+ getNTracks(): Int
+ getLength(): Double
+ getMaxSpeed(): Double
+ setJunction1(Junction*)
+ setJunction2(Junction*)
+ setNTracks(int)
+ setLength(Double)
+ setMaxSpeed(Double)
+ getDetails(): String
Line: undirected edge between two Junctions of the network
• origin: first Junction
• destination: second Junction
• nTracks: number of tracks on the line
• maxSpeed: maximum speed trains can achieve
• getDetails(): provide a textual description of the object
Dispatch: The dispatch information for an individual service
• time: the time of departure from the first station in the ServiceSet
• days: The days of the week this service operates
• trainNumber: This trains's globally unique identifier
• operatesOn(Integer): Returns true if the service is operating on the given day of the week, otherwise False
Dispatch
- time:Integer
- days: Bool[7]
- trainNumbet: Integer
+ getTime(): Integer
+ operatesOn(Integer): bool
+ getTrainNumber(): Integer
<<enumeration>>
ServiceStatus
Effective
PendingApproval
ServiceStatus: the status of a service
• Effective: The service is operating
• PendingApproval: The service is awaiting approval
ServiceStep: one step of a service route
• Junction: The Junction this step refers to
• stopInfo: The stopping information this stop refers to
• service: The service that this step references
• travelMinutes: The time travelled
• isStopping(): If the service is stopping, true, otherwise false
StopInformation: stopping details for a service setp
• waitMinutes: minutes the service waits
• platform: from which platform the passengers access the train
StopData
- waitMinutes:Integer
- platform:String
+ getWait():Integer
+ getPlatform():String
Junction: Junction of the railway network
• getDetails(): provide a textual description of the object
• getDepartures(Time,Time): Compute and return departing services between given time criterion
• getLines(): return a pointer to the connected lines
• getNeighbors(): return the neighboring Junctions
Pathfinding
- requester: Passenger*
- from: Station*
- to: Station*
- after: Option<time_t>
- before: Option<time_t>
- chosenSrt: srtAlg
- results: List<tuple<List<Ticket>, List<Station*>>>
+ constructor(Passenger*, Station*, ...): Pathfinding
+getRequester(): Passenger*
+getFrom(): Station*
+getTo(): Station*
+getAfter(): Option<time_t>
+getBefore(): Option<time_t>
+getResults(): List<tuple<List<Ticket>, List<Station*>>>
- srtPth(): List<tuple<List<Ticket>, List<Station*>>>
+ prchTcks(int)
Pathfinding:
A class of objects representing an in-use pathfinding search. The results of the search are stored each as a list of stations and the tickets that would connect them
+constructor: Explicitly mentioned since it populates the internal fields and computes the possible paths, while other methods modify the results field as requested by the User.
Sets the factory-default in chosenSrt.
- srtPth: Takes the paths created by mkPth and ranks them based on the enum argument, filtering the top results nondestructively. Uses the srtAlg enum to change behaviour based on a pre-set list of supported sorting decisions.
+ getResults: Unlike other getters, calls srtPth to only provide the processed results, without permanently altering the valable (so that the sorting can be repeated without loss of data)
+ prchTcks: Uses getTcks on the same selected path (by index int) to perform UC 12.1
<<enumeration>>
srtAlg
Length
Departure
Arrival
Cost
srtAlg: The sorting algorithm to use
• Lenght: Specified in FR14.1
• Departure: Specified in FR14.2
• Arrival: Specified in FR14.3
• Cost: Specified in FR14.4
1..1
0..*
2..2
0..1